

MileStone-4

Project Title

AI-Based Medical Symptom Analysis & Disease Prediction System

Table of content

Section	Page
1. Project Description	1
2. Exploring Deployment Platform	1
3. Creating/Updating files for deployment	2
4. Deploying	2-3
5. Conclusion	3-4

1. Project Description

During Milestone 4, the MedAssist AI platform was prepared for final deployment and demonstration. The primary objective of this milestone was to integrate the developed **Next.js frontend**, **FastAPI backend**, and trained **CatBoost disease prediction model** into a complete working system.

The frontend was developed using Next.js and provides an interactive interface through which users can enter demographic information and select symptoms. The backend was implemented using FastAPI and is responsible for receiving requests from the frontend, processing the input data, and generating disease predictions using the trained machine learning model.

For deployment, **Vercel** was selected for hosting the Next.js frontend, while **Render** was used to host the Python FastAPI backend along with the trained machine learning model and its required model artifacts.

The final workflow of the deployed system is:

User Input → Vercel Frontend → Render FastAPI Backend → CatBoost Model → Prediction → Vercel Frontend

The milestone also included final model testing and validation, deployment configuration, preparation of documentation and presentation material, and successful end-to-end demonstration of the platform.

2. Exploring Deployment Platform

The deployment requirements of MedAssist AI were analyzed to determine suitable platforms for the frontend and backend.

Since the project consisted of a Next.js frontend and a Python FastAPI backend with a machine learning model, separate deployment platforms were selected.

Vercel – Frontend (<https://vercel.com/>)

Vercel was selected for deploying the Next.js frontend because it provides direct support for Next.js applications and simplifies the build and deployment process.

The following aspects were explored:

- Next.js project deployment
- Git repository integration
- Automatic build and deployment
- Environment variable configuration
- Production deployment
- Frontend-to-backend API communication

Render – Backend and Model (<https://render.com/>)

Render was selected for deploying the Python FastAPI backend and the trained machine learning model.

The following aspects were explored:

- Python web-service deployment
- FastAPI server configuration
- Dependency installation
- Start command configuration
- Environment variables
- Model file availability
- API accessibility through a public URL

The deployment architecture was therefore divided into two services:

Vercel → Next.js Frontend

Render → FastAPI Backend + CatBoost Model

This architecture allowed the frontend and machine learning backend to be independently deployed while maintaining communication through the REST API.

3. Creating/Updating Files for Deployment

Before deployment, the project files were reviewed and updated to make the frontend and backend compatible with their respective deployment platforms.

Frontend – Next.js

The Next.js frontend was prepared for deployment on **Vercel**. The frontend API configuration was updated so that requests were sent to the publicly deployed Render backend rather than the local development server.

Environment variables were configured to store the backend API URL. This allowed the frontend to communicate with the deployed FastAPI service without hard-coding local development addresses.

The Next.js project configuration and dependencies were also reviewed to ensure that the application could be successfully built and deployed on Vercel.

Backend – FastAPI and Machine Learning Model

The Python backend was prepared for deployment on **Render**. The FastAPI application was configured to run as a web service and accept requests from the deployed frontend.

The trained CatBoost model and required supporting files were included with the backend. These included:

- Trained CatBoost disease prediction model
- Label encoder
- Feature-column configuration
- Required Python dependencies

The API endpoints were tested before deployment. The health-check endpoint was used to verify that the backend was running, while the prediction endpoint was tested with sample symptom inputs.

Deployment configuration files and dependency files were also updated to define the Python environment, required packages, and server start command.

4. Deploying Frontend and Backend

The final deployment was performed using **Vercel for the frontend** and **Render for the backend and machine learning model**.

Frontend Deployment on Vercel

The Next.js application was connected to the project repository and deployed through Vercel. The required environment variables were configured so that the frontend could access the deployed FastAPI backend.

After deployment, the Vercel application was opened and tested to verify that the user interface loaded correctly and that all required components were functioning.

Backend and Model Deployment on Render

The FastAPI backend was deployed as a Python web service on Render. The required dependencies were installed and the application start command was configured.

The trained CatBoost model and its supporting files were made available to the backend so that the model could be loaded when the application started.

The backend health endpoint was tested after deployment to verify that the service was running successfully.

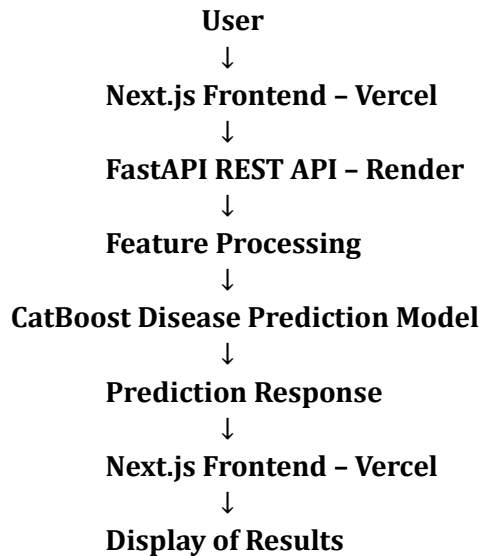
The prediction API was then tested using sample requests. The tests verified that the backend could:

1. Receive symptom and demographic information.
2. Process the submitted features.
3. Load and execute the trained CatBoost model.
4. Generate disease predictions.
5. Return prediction results through the API.

End-to-End Testing

After both services were deployed, communication between Vercel and Render was tested.

The complete workflow was:



The deployed system was successfully tested from user input through final prediction output. Model testing and validation were also completed to verify the reliability and consistency of the prediction system.

5. Conclusion

Milestone 4 completed the final development, deployment, testing, and demonstration phase of the MedAssist AI project.

The **Next.js frontend was successfully deployed on Vercel**, while the **FastAPI backend and trained CatBoost disease prediction model were deployed on Render**. The two services were successfully connected through the REST API.

The deployed model was tested using appropriate test cases, and the complete application workflow was verified from symptom input to disease prediction. The deployment configuration, environment variables, API communication, and model integration were also verified.

The required project documentation and presentation materials were prepared to describe the system architecture, technologies used, machine learning model, deployment process, and project results.

Finally, an end-to-end demonstration of the deployed MedAssist AI platform was successfully completed. The demonstration confirmed that users could access the Next.js application, submit symptoms, communicate with the Render-hosted FastAPI backend, obtain predictions from the trained CatBoost model, and view the results through the frontend.

Therefore, Milestone 4 successfully achieved its objective of delivering a **fully deployed, tested, validated, documented, and demonstrable MedAssist AI platform using Vercel and Render**.

Link: <https://healthassistai-one.vercel.app/>

